

Coding First Priority

Rule used: Top 10 = highest-frequency coding/theory questions. Top 25 = Q11–Q25 continuation. Top 50 = Q26–Q50 continuation. No repeats across sections.

Top 10 – Most Important SQL Interview Questions

Q1: SQL Joins – INNER, LEFT, RIGHT, FULL

Answer (from given info): Joins combine rows from two tables based on a matching condition. INNER returns only matches in both tables. LEFT returns all left rows plus matches from right, NULL when absent. RIGHT is the mirror. FULL returns all rows from either table, NULL where missing.

Polished answer: In interviews, explain joins as row-combining operations based on keys. Then write syntax for each type and mention NULL behavior.

TL;DR: INNER = intersection; LEFT = all left; RIGHT = all right; FULL = all rows.

Key mappings: ON , matching keys, NULL padding, Venn diagrams.

```
SELECT *  
FROM A  
INNER JOIN B ON A.id = B.a_id;
```

```
SELECT *  
FROM A  
LEFT JOIN B ON A.id = B.a_id;
```

```
SELECT *  
FROM A  
RIGHT JOIN B ON A.id = B.a_id;
```

```
SELECT *  
FROM A  
FULL JOIN B ON A.id = B.a_id;
```

Q2: WHERE vs HAVING

Answer (from given info): WHERE filters individual rows before grouping/aggregation, so it cannot use aggregate functions. HAVING filters groups after GROUP BY, so it is used for aggregate conditions.

Polished answer: Use WHERE to reduce raw rows early. Use HAVING when the condition depends on COUNT, SUM, AVG, etc.

TL;DR: WHERE filters rows; HAVING filters groups.

Key mappings: GROUP BY , COUNT (*) , aggregate filtering, query order.

```
SELECT customer_id, COUNT(*) AS orders_2025
FROM orders
WHERE order_date >= '2025-01-01' AND order_date < '2026-01-01'
GROUP BY customer_id
HAVING COUNT(*) > 5;
```

Q3: GROUP BY with Aggregate Functions

Answer (from given info): GROUP BY arranges identical data into groups. It is typically used with aggregate functions like COUNT, SUM, AVG, MIN, MAX to calculate per group.

Polished answer: GROUP BY collapses rows by a key. Every selected non-aggregated column must appear in GROUP BY.

TL;DR: GROUP BY = group rows; aggregates = summarize each group.

Key mappings: COUNT() , SUM() , AVG() , MIN() , MAX() .

```
SELECT department_id,
       COUNT(*) AS employee_count,
       AVG(salary) AS avg_salary
FROM Employees
GROUP BY department_id;
```

Q4: Running Total per Product

Answer (from given info): Running total is calculated using SUM() window function with OVER(). It adds each row's value to the cumulative total while keeping individual rows.

Polished answer: Use PARTITION BY product_id and ORDER BY sale_date. This is a classic window-function interview question.

TL;DR: SUM(amount) OVER (PARTITION BY product_id ORDER BY sale_date) .

Key mappings: window function, cumulative sum, partition, order.

```
SELECT
    product_id,
    sale_date,
    amount,
    SUM(amount) OVER (
        PARTITION BY product_id
        ORDER BY sale_date
    ) AS running_total
FROM Sales;
```

Q5: ROW_NUMBER vs RANK vs DENSE_RANK

Answer (from given info): ROW_NUMBER assigns a unique number to each row. RANK assigns the same rank to duplicates and skips the next rank. DENSE_RANK assigns the same rank to duplicates but does not skip.

Polished answer: Use ROW_NUMBER for pagination/dedup. Use RANK for competition ranking with gaps. Use DENSE_RANK for ranking without gaps.

TL;DR: ROW_NUMBER = unique; RANK = gaps; DENSE_RANK = no gaps.

Key mappings: OVER() , ORDER BY , duplicates, pagination.

```
SELECT name, salary,
    ROW_NUMBER() OVER (ORDER BY salary DESC) AS row_num,
    RANK() OVER (ORDER BY salary DESC) AS rank_num,
    DENSE_RANK() OVER (ORDER BY salary DESC) AS dense_rank_num
FROM Employees;
```

Q6: Correlated Subquery

Answer (from given info): A correlated subquery references columns from the outer query. It executes once for each row processed by the outer query.

Polished answer: Use it when the inner query depends on the current outer row. Classic example: employees earning more than their department average.

TL;DR: Inner query depends on outer row; runs row-by-row.

Key mappings: outer reference, department average, WHERE e.salary > (...).

```
SELECT e.employee_id, e.name, e.salary, e.department_id
FROM Employees e
WHERE e.salary > (
```

```
SELECT AVG(e2.salary)
FROM Employees e2
WHERE e2.department_id = e.department_id
);
```

Q7: EXISTS vs NOT EXISTS vs IN + Anti-Join

Answer (from given info): EXISTS returns TRUE if the subquery returns at least one row. NOT EXISTS returns TRUE if no rows are returned. IN checks whether a value exists in a list or subquery result. EXISTS/NOT EXISTS stop when a match is found; IN compares against all values.

Polished answer: Use EXISTS for existence checks, NOT EXISTS for anti-joins, and IN for small static lists.

TL;DR: EXISTS = at least one; NOT EXISTS = none; IN = value in list.

Key mappings: anti-join, `SELECT 1`, correlated subquery, NULL-safe behavior.

```
SELECT CustomerID
FROM Customers c
WHERE EXISTS (
    SELECT 1
    FROM Orders o
    WHERE o.CustomerID = c.CustomerID
);
```

```
SELECT c.CustomerID, c.CustomerName
FROM Customers c
WHERE NOT EXISTS (
    SELECT 1
    FROM Orders o
    WHERE o.CustomerID = c.CustomerID
);
```

Q8: Delete Duplicate Rows Keeping First Row

Answer (from given info): Use a self-join/delete pattern where duplicate names exist and keep the row with the smallest Id.

Polished answer: In MySQL-style self-join, delete M1 when another row M2 has the same name and a smaller Id. In modern SQL, use ROW_NUMBER() in a CTE.

TL;DR: Keep min Id per duplicate key.

Key mappings: self-join delete, `ROW_NUMBER()`, duplicate cleanup.

```
DELETE M1
FROM managers M1, managers M2
WHERE M2.Name = M1.Name
      AND M1.Id > M2.Id;
```

```
WITH CTE AS (
    SELECT *,
           ROW_NUMBER() OVER (PARTITION BY Name ORDER BY Id) AS rn
    FROM managers
)
DELETE FROM CTE WHERE rn > 1;
```

Q9: COALESCE for NULL Names

Answer (from given info): COALESCE returns the first non-NULL value from a list of expressions.

Polished answer: Use COALESCE to pick First_Name, then Second_Name, then Last_Name when earlier names are NULL.

TL;DR: COALESCE = first non-NULL.

Key mappings: NULL handling, fallback value, `COALESCE(FName, SName, LName)` .

```
SELECT ID,
       COALESCE(FName, SName, LName) AS Name
FROM employees;
```

Q10: Recursive CTE for Hierarchical Data

Answer (from given info): SQL uses recursive CTEs to retrieve hierarchical or tree-structured data.

Polished answer: Anchor member gets roots. Recursive member joins child rows to the CTE. Use UNION ALL.

TL;DR: Recursive CTE = anchor + recursive join.

Key mappings: `WITH RECURSIVE` , `ParentID` , `Depth` , hierarchy.

```
WITH RecursiveCTE (ID, ParentID, Depth) AS (
    SELECT ID, ParentID, 1
    FROM Categories
    WHERE ParentID IS NULL
```

UNION ALL

```
SELECT c.ID, c.ParentID, r.Depth + 1
FROM Categories c
JOIN RecursiveCTE r
    ON c.ParentID = r.ID
)
SELECT * FROM RecursiveCTE;
```

Top 25 – Continuation Q11–Q25

Q11: LAG and LEAD

Answer (from given info): LAG returns previous row value; LEAD returns next row value. Used to compare consecutive rows without self-join.

Polished answer: Great for month-over-month differences, trend analysis, and consecutive row comparisons.

TL;DR: LAG = previous; LEAD = next.

Key mappings: window function, `OVER (ORDER BY ...)`, trend.

```
SELECT sale_date, amount,
       LAG(amount) OVER (ORDER BY sale_date) AS prev_amount,
       LEAD(amount) OVER (ORDER BY sale_date) AS next_amount
FROM Sales;
```

Q12: CASE Statement

Answer (from given info): CASE implements conditional logic. It returns a value based on first true condition, with optional ELSE.

Polished answer: Use CASE for bucketing, labels, and conditional aggregation.

TL;DR: CASE = SQL if-else.

Key mappings: `WHEN`, `THEN`, `ELSE`, `END`.

```
SELECT ID,
       CASE
           WHEN Salary > 100000 THEN 'High'
           WHEN Salary BETWEEN 50000 AND 100000 THEN 'Medium'
```

```
ELSE 'Low'  
END AS SalaryLevel  
FROM Employees;
```

Q13: PIVOT

Answer (from given info): PIVOT transforms rows into columns for reporting.

Polished answer: Use PIVOT with an aggregate and FOR ... IN(...) to turn year/month rows into columns.

TL;DR: PIVOT = row-to-column summary.

Key mappings: `SUM(Amount)` , `FOR SaleYear IN (...)` , reporting.

```
SELECT ProductID, [2021], [2022]  
FROM (  
    SELECT ProductID, YEAR(SaleDate) AS SaleYear, Amount  
    FROM Sales  
    ) AS Source  
PIVOT (  
    SUM(Amount)  
    FOR SaleYear IN ([2021], [2022])  
    ) AS PivotTable;
```

Q14: UNION vs UNION ALL + Set Operations

Answer (from given info): UNION combines and removes duplicates. UNION ALL keeps duplicates and is faster. INTERSECT returns common rows. EXCEPT/MINUS returns rows in first query not in second.

Polished answer: Use UNION ALL unless deduplication is required. Set operations require same number/order/compatible data types.

TL;DR: UNION dedups; UNION ALL keeps all; INTERSECT common; EXCEPT difference.

Key mappings: set operators, duplicate removal, column compatibility.

```
SELECT name FROM Students  
UNION  
SELECT name FROM Contacts;
```

```
SELECT name FROM Students  
UNION ALL  
SELECT name FROM Contacts;
```

```
SELECT name FROM Students
```

```
INTERSECT
SELECT name FROM Contacts;

SELECT name FROM Students
EXCEPT
SELECT name FROM Contacts;
```

Q15: SELF JOIN

Answer (from given info): SELF JOIN joins a table with itself. Useful for employee-manager or parent-child relationships. Use aliases.

Polished answer: Use two aliases for same table and join manager ID to employee ID.

TL;DR: Self join = table joined to itself.

Key mappings: aliases, `ManagerID = EmployeeID`, hierarchy.

```
SELECT e.EmployeeName,
       m.EmployeeName AS Manager
FROM Employees e
LEFT JOIN Employees m
  ON e.ManagerID = m.EmployeeID;
```

Q16: Pattern Matching with LIKE

Answer (from given info): LIKE uses wildcards: `%` matches zero or more characters; `_` matches exactly one character.

Polished answer: Use `%` for any length, `_` for fixed position/length. Combine with NOT for exclusion.

TL;DR: `%` = any chars; `_` = one char.

Key mappings: wildcard, position, length, `NOT LIKE`.

```
SELECT * FROM students WHERE first_name LIKE 'K%';
SELECT * FROM students WHERE first_name LIKE '%Q%';
SELECT * FROM students WHERE first_name LIKE '__K%';
SELECT * FROM students WHERE first_name LIKE '____';
```

Q17: Handle Duplicates Without DISTINCT

Answer (from given info): Use GROUP BY or ROW_NUMBER() to eliminate duplicates without DISTINCT.

Polished answer: GROUP BY for simple dedup with aggregate. ROW_NUMBER() for keeping one full row per partition.

TL;DR: GROUP BY or ROW_NUMBER() instead of DISTINCT.

Key mappings: PARTITION BY , RowNum = 1 , dedup.

```
SELECT Column1, MAX(Column2)
FROM TableName
GROUP BY Column1;
```

```
WITH CTE AS (
    SELECT Column1, Column2,
           ROW_NUMBER() OVER (
               PARTITION BY Column1
               ORDER BY Column2
           ) AS RowNum
    FROM TableName
)
SELECT *
FROM CTE
WHERE RowNum = 1;
```

Q18: MERGE Statement

Answer (from given info): MERGE combines INSERT, UPDATE, DELETE to synchronize target and source tables.

Polished answer: Use MERGE for upserts. Mention concurrency risks in SQL Server.

TL;DR: MERGE = insert/update/delete in one statement.

Key mappings: WHEN MATCHED , WHEN NOT MATCHED , source/target.

```
MERGE INTO TargetTable T
USING SourceTable S
ON T.ID = S.ID
WHEN MATCHED THEN
    UPDATE SET T.Value = S.Value
WHEN NOT MATCHED THEN
    INSERT (ID, Value) VALUES (S.ID, S.Value);
```

Q19: CROSS JOIN vs INNER JOIN

Answer (from given info): CROSS JOIN returns Cartesian product; no join condition. INNER JOIN returns only matching rows and requires ON.

Polished answer: CROSS JOIN row count = left rows × right rows. INNER JOIN row count depends on matches.

TL;DR: CROSS = all combinations; INNER = matches only.

Key mappings: Cartesian product, **ON**, row multiplication.

```
SELECT stu.name, sub.subject
FROM students AS stu
CROSS JOIN subjects AS sub;

SELECT *
FROM A
INNER JOIN B ON A.id = B.a_id;
```

Q20: Subquery Basics – Non-Correlated

Answer (from given info): A subquery is a query within another query. Non-correlated subquery can run independently and its output is substituted into main query.

Polished answer: Use non-correlated subqueries with IN, NOT IN, comparison operators.

TL;DR: Inner query runs once; result used by outer query.

Key mappings: nested query, **IN**, independent subquery.

```
SELECT name, email, mob, address
FROM contacts
WHERE roll_no IN (
    SELECT roll_no
    FROM students
    WHERE subject = 'Maths'
);
```

Q21: CTE (Common Table Expression)

Answer (from given info): CTE is a temporary named result set created with WITH. Exists during one SQL statement. Simplifies complex queries and improves readability.

Polished answer: Use CTEs to break complex logic into readable steps. They are not always materialized.

TL;DR: CTE = named temporary result for one statement.

Key mappings: `WITH` , readability, avoiding repeated subqueries.

```
WITH SalesCTE AS (  
    SELECT product_id, SUM(amount) AS total_sales  
    FROM Sales  
    GROUP BY product_id  
)  
SELECT *  
FROM SalesCTE  
WHERE total_sales > 1000;
```

Q22: Window Functions General

Answer (from given info): Window functions calculate across related rows while keeping each row separate. Used for running totals, rankings, moving averages.

Polished answer: Window functions use `OVER()`. They do not collapse rows like `GROUP BY`.

TL;DR: Window function = calculation without losing row detail.

Key mappings: `OVER()` , `PARTITION BY` , `ORDER BY` , running totals.

```
SELECT Name, Salary,  
       SUM(Salary) OVER (ORDER BY Salary) AS RunningTotal  
FROM Employees;
```

Q23: COUNT vs SUM and COUNT on NULLs

Answer (from given info): `COUNT` counts rows or non-NULL values. `SUM` adds numeric values. `COUNT(column)` ignores NULLs; `COUNT()` counts all rows.

Polished answer: `COUNT()` counts rows. `COUNT(col)` counts non-NULL col. `SUM` ignores NULLs.

TL;DR: `COUNT(*)` = all rows; `COUNT(col)` = non-NULL; `SUM` = numeric total.

Key mappings: NULL handling, row count, numeric aggregation.

```
SELECT COUNT(*) FROM Orders;  
SELECT COUNT(column_name) FROM Orders;  
SELECT SUM(TotalAmount) FROM Orders;
```

Q24: Employees Hired in Last N Months

Answer (from given info): Use `TIMESTAMPDIFF(month, Hiredate, current_date())` and filter between 1 and 5.

Polished answer: Use `TIMESTAMPDIFF` for month difference. Filter and order by Hiredate desc.

TL;DR: `TIMESTAMPDIFF` + `BETWEEN` for last N months.

Key mappings: date functions, `CURRENT_DATE`, month difference.

```
SELECT *,
        TIMESTAMPDIFF(MONTH, Hiredate, CURRENT_DATE()) AS DiffMonth
FROM employees
WHERE TIMESTAMPDIFF(MONTH, Hiredate, CURRENT_DATE())
      BETWEEN 1 AND 5
ORDER BY Hiredate DESC;
```

Q25: Create Empty Table with Same Structure

Answer (from given info): Use `SELECT INTO` with a false WHERE clause, e.g., `WHERE 1 = 2`.

Polished answer: This creates a duplicate structure but inserts no rows.

TL;DR: `SELECT * INTO new_table FROM old_table WHERE 1=2`.

Key mappings: table clone, `INTO`, false predicate.

```
SELECT * INTO Students_copy
FROM Students
WHERE 1 = 2;
```

Top 50 – Continuation Q26–Q50

Q26: Clustered vs Non-Clustered Index

Answer (from given info): Clustered index stores table rows in physical order of index key; only one per table. Non-clustered stores index separately with pointers; multiple allowed.

Polished answer: Clustered = range/sorting friendly. Non-clustered = filtering/joins/lookups.

TL;DR: Clustered = physical order; non-clustered = separate pointer structure.

Key mappings: primary key, range query, pointer, one vs many.

```
CREATE CLUSTERED INDEX idx_emp_id ON Employees(EmployeeID);  
CREATE NONCLUSTERED INDEX idx_lastname ON Employees(LastName);
```

Q27: CREATE INDEX and Query Performance

Answer (from given info): Indexing helps database find data without scanning whole table.

Polished answer: Create index on columns used in WHERE, JOIN, ORDER BY.

TL;DR: Index = faster lookup, extra storage/write cost.

Key mappings: CREATE INDEX, WHERE, scan vs seek.

```
CREATE INDEX idx_lastname ON Employees(LastName);
```

```
SELECT *  
FROM Employees  
WHERE LastName = 'Smith';
```

Q28: PRIMARY KEY vs UNIQUE

Answer (from given info): PRIMARY KEY = UNIQUE + NOT NULL, only one per table, default foreign key target. UNIQUE allows NULLs and multiple per table.

Polished answer: PK identifies row. UNIQUE enforces distinctness but allows NULL in many DBs.

TL;DR: PK = unique + not null + one; UNIQUE = distinct + multiple + nullable.

Key mappings: constraints, NULL, foreign key.

```
CREATE TABLE Students (  
    ID INT NOT NULL PRIMARY KEY,  
    Email VARCHAR(255) UNIQUE  
);
```

Q29: FOREIGN KEY and Referential Integrity

Answer (from given info): Foreign key references primary key of another table and ensures values exist in parent table, preventing orphan records.

Polished answer: Child table references parent. Use ON DELETE/UPDATE CASCADE for automatic actions.

TL;DR: FK = referential integrity between child and parent.

Key mappings: parent/child, REFERENCES , orphan prevention.

```
CREATE TABLE Students (  
    ID INT PRIMARY KEY,  
    LibraryID INT,  
    FOREIGN KEY (LibraryID) REFERENCES Library(LibraryID)  
);
```

Q30: SQL Constraints

Answer (from given info): Constraints enforce rules: NOT NULL, UNIQUE, PRIMARY KEY, FOREIGN KEY, CHECK, DEFAULT.

Polished answer: Constraints maintain data integrity automatically.

TL;DR: Constraints = rules for valid data.

Key mappings: integrity, validation, default.

```
CREATE TABLE Employees (  
    ID INT PRIMARY KEY,  
    Salary DECIMAL(10,2) CHECK (Salary > 0),  
    Status VARCHAR(20) DEFAULT 'Active',  
    Email VARCHAR(255) UNIQUE  
);
```

Q31: Normalization and Normal Forms

Answer (from given info): Normalization minimizes redundancy and prevents anomalies. 1NF atomic values/no repeating groups. 2NF no partial dependency. 3NF no transitive dependency. BCNF every determinant is candidate key. 4NF removes multi-valued dependencies. 5NF removes join dependencies.

Polished answer: In interviews, explain 1NF → 2NF → 3NF → BCNF with dependency logic.

TL;DR: 1NF atomic; 2NF no partial; 3NF no transitive; BCNF superkey.

Key mappings: redundancy, anomalies, functional dependency.

Q32: Denormalization

Answer (from given info): Denormalization combines normalized tables for performance. Used when joins slow reads and redundancy is acceptable.

Polished answer: Denormalize for reporting/read-heavy systems; accept update complexity.

TL;DR: Denormalization = intentional redundancy for speed.

Key mappings: read performance, redundancy, joins.

Q33: ACID Properties

Answer (from given info): Atomicity all-or-nothing; Consistency valid rules; Isolation independent transactions; Durability committed changes persist.

Polished answer: ACID guarantees reliable transactions.

TL;DR: Atomicity, Consistency, Isolation, Durability.

Key mappings: transaction, rollback, commit, failure.

Q34: Isolation Levels

Answer (from given info): Read Uncommitted allows dirty reads. Read Committed prevents dirty reads. Repeatable Read prevents dirty and non-repeatable reads. Serializable prevents dirty, non-repeatable, phantom reads.

Polished answer: Higher isolation = more consistency, less concurrency.

TL;DR: Serializable strongest; Read Uncommitted weakest.

Key mappings: dirty read, phantom read, non-repeatable read.

Q35: DELETE vs TRUNCATE vs DROP

Answer (from given info): DELETE removes rows with WHERE, DML, rollback possible. TRUNCATE removes all rows, DDL, faster, no WHERE. DROP removes table structure and all data.

Polished answer: DELETE = row-level; TRUNCATE = all rows; DROP = object removal.

TL;DR: DELETE rows; TRUNCATE all rows; DROP table.

Key mappings: DML, DDL, rollback, structure.

```
DELETE FROM Candidates WHERE CandidateId > 1000;
TRUNCATE TABLE Candidates;
DROP TABLE Candidates;
```

Q36: DDL vs DML

Answer (from given info): DDL defines/modifies structure: CREATE, ALTER, DROP, TRUNCATE. DML manages data: SELECT, INSERT, UPDATE, DELETE.

Polished answer: DDL changes schema; DML changes rows.

TL;DR: DDL = structure; DML = data.

Key mappings: CREATE, ALTER, INSERT, UPDATE.

```
CREATE TABLE T (id INT);  
ALTER TABLE T ADD name VARCHAR(50);  
INSERT INTO T VALUES (1, 'A');  
UPDATE T SET name = 'B' WHERE id = 1;
```

Q37: Stored Procedures

Answer (from given info): Precompiled SQL statements stored in database. Accept parameters, perform logic, return values/result sets. Improve performance and maintainability.

Polished answer: Centralize business logic. Mention security and memory trade-offs.

TL;DR: Stored procedure = reusable precompiled SQL block.

Key mappings: parameters, business logic, precompiled.

```
CREATE PROCEDURE FetchAllStudents()  
BEGIN  
    SELECT * FROM myDB.students;  
END;
```

Q38: Triggers

Answer (from given info): Triggers automatically execute on INSERT, UPDATE, DELETE. Used for consistency, business rules, audit logs.

Polished answer: BEFORE/AFTER triggers depend on DB. Use carefully; can hide logic.

TL;DR: Trigger = automatic action on table event.

Key mappings: audit, business rule, INSERT/UPDATE/DELETE.

```
CREATE TRIGGER trg_audit  
AFTER INSERT ON Orders  
FOR EACH ROW
```

```
INSERT INTO AuditLog(order_id, action)
VALUES (NEW.id, 'INSERT');
```

Q39: Cursors

Answer (from given info): Cursor retrieves/manipulates rows one at a time. Types include STATIC, DYNAMIC, FORWARD_ONLY, KEYSET.

Polished answer: Use cursors only when set-based SQL is not suitable. They are slower.

TL;DR: Cursor = row-by-row processing.

Key mappings: DECLARE, OPEN, FETCH, CLOSE, DEALLOCATE.

```
DECLARE db_cursor CURSOR FOR
SELECT name FROM students;

OPEN db_cursor;
FETCH NEXT FROM db_cursor INTO @name;
CLOSE db_cursor;
DEALLOCATE db_cursor;
```

Q40: Local vs Global Temporary Tables

Answer (from given info): Local temp table `#Temp` exists only in session and is dropped when session ends. Global temp table `##GlobalTemp` visible to all sessions and dropped when all referencing sessions close.

Polished answer: Use temp tables for intermediate results.

TL;DR: `#` local; `##` global.

Key mappings: session scope, temp storage.

```
CREATE TABLE #LocalTemp (ID INT);
CREATE TABLE ##GlobalTemp (ID INT);
```

Q41: Standard View vs Materialized View

Answer (from given info): Standard view is virtual and does not store data; query runs each time. Materialized view stores query result physically and needs refresh.

Polished answer: Use materialized views for fast reporting/aggregations. Refresh ON DEMAND or ON

COMMIT.

TL;DR: View = virtual; materialized view = stored snapshot.

Key mappings: refresh, precomputed, reporting.

```
REFRESH MATERIALIZED VIEW my_view;
```

Q42: Horizontal vs Vertical Partitioning

Answer (from given info): Horizontal splits rows; vertical splits columns. Horizontal reduces rows scanned; vertical separates frequently/rarely used data.

Polished answer: Horizontal by region/year; vertical by large text columns separately.

TL;DR: Horizontal = rows; vertical = columns.

Key mappings: partition key, performance, manageability.

Q43: Sharding vs Partitioning

Answer (from given info): Sharding splits database into independent databases across servers. Partitioning splits a table within same database.

Polished answer: Sharding = horizontal scaling; partitioning = performance/data management.

TL;DR: Sharding = across servers; partitioning = within database.

Key mappings: distributed data, scalability, region.

Q44: Optimize a Slow Query

Answer (from given info): Use EXPLAIN, add indexes/update statistics, avoid functions on columns, filter early, avoid SELECT *, optimize joins, rewrite queries, use pagination/caching/partitioning.

Polished answer: Measure first with EXPLAIN, then index and rewrite.

TL;DR: EXPLAIN → index → filter early → rewrite.

Key mappings: execution plan, index, statistics, pagination.

```
EXPLAIN ANALYZE
SELECT customer_id, COUNT(*)
FROM orders
WHERE order_date >= '2025-01-01'
GROUP BY customer_id;
```

Q45: Prevent SQL Injection

Answer (from given info): Use parameterized queries, validate/sanitize input, avoid dynamic SQL string concatenation, least-privilege accounts, secure stored procedures.

Polished answer: Parameterization is the main defense. Never concatenate user input into SQL.

TL;DR: Prepared statements + validation + least privilege.

Key mappings: SQL injection, prepared statement, sanitization.

```
-- Safe concept
SELECT * FROM users WHERE email = ?;
```

Q46: OLTP vs OLAP

Answer (from given info): OLTP handles simple frequent transactions, optimized for read/write, normalized schema. OLAP handles complex analysis, read-heavy, denormalized star/snowflake.

Polished answer: OLTP = day-to-day operations; OLAP = analytics/reporting.

TL;DR: OLTP = transactions; OLAP = analytics.

Key mappings: normalization, aggregation, data warehouse.

Q47: Deadlock vs Blocking vs Livelock

Answer (from given info): Blocking = one transaction waits for lock release. Deadlock = circular wait, needs detection/rollback. Livelock = transactions keep responding but no progress.

Polished answer: Prevent deadlocks with consistent resource ordering, short transactions, proper indexes, isolation tuning.

TL;DR: Blocking resolves; deadlock needs rollback; livelock no progress.

Key mappings: lock contention, retry, isolation.

Q48: Database Snapshot

Answer (from given info): Read-only copy of database at a point in time. Used for reporting, point-in-time recovery, testing.

Polished answer: Snapshot gives consistent dataset without affecting original.

TL;DR: Snapshot = point-in-time read-only copy.

Key mappings: recovery, reporting, testing.

```
CREATE DATABASE MySnapshot ON
(
    NAME = MyDatabase_Data,
    FILENAME = 'C:\Snapshots\MyDatabase_Snapshot.ss'
)
AS SNAPSHOT OF MyDatabase;
```

Q49: Scalar Functions

Answer (from given info): Scalar functions operate on individual values and return a single value.

Examples: LEN, ROUND, CONVERT.

Polished answer: Use for formatting/conversion. They are not aggregate functions.

TL;DR: Scalar = one input row value → one output value.

Key mappings: LEN, ROUND, CONVERT.

```
SELECT LEN('SQL'), ROUND(123.456, 2), CONVERT(VARCHAR, GETDATE(), 120);
```

Q50: SQL Command Types and Operators

Answer (from given info): DDL: CREATE, ALTER, DROP, TRUNCATE. DML: SELECT, INSERT, UPDATE, DELETE. DCL: GRANT, REVOKE. TCL: COMMIT, ROLLBACK, SAVEPOINT. Operators include arithmetic, comparison, logical, set, special, concatenation.

Polished answer: Know command families and operator categories. This is often a warm-up theory question.

TL;DR: DDL, DML, DCL, TCL + operators.

Key mappings: CREATE/ALTER, SELECT/INSERT, GRANT/REVOKE, COMMIT/ROLLBACK.

```
-- DDL
CREATE TABLE T (id INT);
-- DML
SELECT * FROM T;
-- TCL
COMMIT;
-- DCL
GRANT SELECT ON T TO user1;
```